Project Implementation II
Advanced Deep Learning Modes

Sidharth Gupta

01 March 2026

Using cGAN to perform Class-Conditional Image Generation on CIFAR-10

Abstract

This report presents the implementation of cGAN [3] to perform class-conditional image generation on CIFAR-10 [2]. The model and the training loop are made from scratch to have more control over training. Label smoothing was done to prevent the discriminator from being too confident and not letting the generator learn. Model was trained over 60 epochs with checkpoints stored after 10 epochs to choose the best state acquired between those epochs. The model is then evaluated on the FID Score and visualized in terms of training loop losses and images generated by the best model parameters. The set target FID Score was not achieved due to time constraints, lesser experimentation with cGAN structure and the low resolution of CIFAR-10 images, only 32x32. For these reasons and 8-hour time constraint an FID score of about 103.77 was achieved.

1 Introduction

In real-world datasets class imbalance is a real problem, i.e., some classes have lesser number of samples than others which can cause models to be biased towards the majority classes. Generative Adversarial Networks (GANs) [1] have a structure that trains a discriminator and a generator side by side, like a competition, where the discriminator tells whether an image is fake or real and the generator tries to generate images from random noise aimed at fooling the discriminator into labelling them as real images. With this competition, if balanced, both the discriminator and generator become strong. A Conditional GAN (cGAN) [3] extends this by conditioning both networks on a class label, enabling the generator to produce images of a specific class on demand. Once the generator is trained enough, it can be used to generate images to make up for the class-imbalance in certain datasets, then on which other models can be trained with perfect class balance. The aim of this project is to try to train a decent generator on CIFAR-10 dataset [2] as a proof of concept. The following sections present the literature survey, methodology, results, challenges and possible improvements.

2 Literature Survey

In 2014, Goodfellow et al. introduced the adversarial training framework (GAN) [1], i.e., the concept of adversarial training scheme between generator and discriminator. This was further extended by Mirza et al. in 2014 who gave the Conditional GAN (cGAN) [3] idea of feeding class labels to both the generator and the discriminator that enabled on-demand (controlled) generation. Further, replacing the dense/fully connected layers with convolutional layers was introduced by Radford et al. [4] as DCGAN in 2015 along with training guidelines such as BatchNorm and LeakyReLU, significantly stablising GAN training on image data. All this put together in this project, explores the possibility of training the generator to the extent to make it good enough to generate samples for an under-sampled class in a dataset.

3 Methodology

3.1 Dataset and Preprocessing

The CIFAR-10 [2] dataset is used to train the model. It consists of 60000 32x32 RGB images of 10 classes (6000 images each), namely, airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck. The images are divided uniformly into train and test sets with 50000 as training images and 10000 as test images. The annotations for these images are just the class labels.

The training images are preprocessed, converted to tensor using torchvision's ToTensor() and normalized to [-1,1] for tanh activation at the end of the generator. The labels are one-hot encoded as they are to be supplied to both the generator and the discriminator.

DataLoader is used to load the data in batches of 64, with shuffling enabled for training set to ensure varied batches during training across epochs.

3.2 Model Architecture

3.2.1 Generator

The generator receives a concatenated input as: noise vector (length 100) + labels (10) = 110 concatenated vector. This vector goes through a Linear Layer and is reshaped to (512, 4, 4). Then three ConvTranspose2D layers upsample the input to 32x32 with ReLU activation and BatchNorm layer after the first and second convolutional layer. Tanh activation is given at the output layer for [-1,1].

3.2.2 Discriminator

Discriminator gets input as image (3 channels) + labels (10) = 13 channels, 32x32. Then three Conv2d layers downsample the input to 4x4 with LeakyReLU(0.2) activation and BatchNorm layer only after second and third convolutional layers. The output from Conv layers is then flattened and supplied to Linear layer with sigmoid activation (real or fake binary output)

3.3 Loss Function

Binary Cross-Entropy (BCE) loss is used as the output of the discriminator is if the image is binary; real or fake.

3.4 Training Procedure

Separate optimizers are used for the generator and discriminator, both Adam with learning rates 0.0002 and 0.0001, respectively, with betas=(0.5, 0.999) for both. For training, label smoothing is done, i.e., real_labels x 0.9 so as to not let the discriminator become too confident and have its loss vanish, not letting the generator learn.

In one training loop, first the discriminator is trained with its loss taken as BCEloss(prediction, real_labels) and then the generator is trained on a loss that looks like BCEloss(predictions, real_labels) as it wants to fool the discriminator into thinking the images supplied by it are real.

The batch size is 64 and training is done for 60 epochs with checkpoints after 10 epochs so model performance can be compared at 6 different points. Training was performed on Google Colab using a T4 GPU, with each epoch taking approximately one minute.

4 Results and Evaluation

4.1 Quantitative Performance

FID score measures the statistical difference between the distribution of real images and the generated images. It captures both quality and diversity. The lower the score, the better the

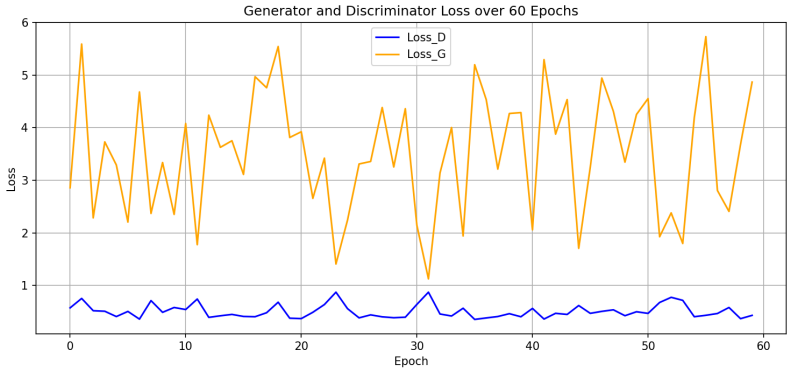

Figure 1: Generator vs Discriminator Training loss

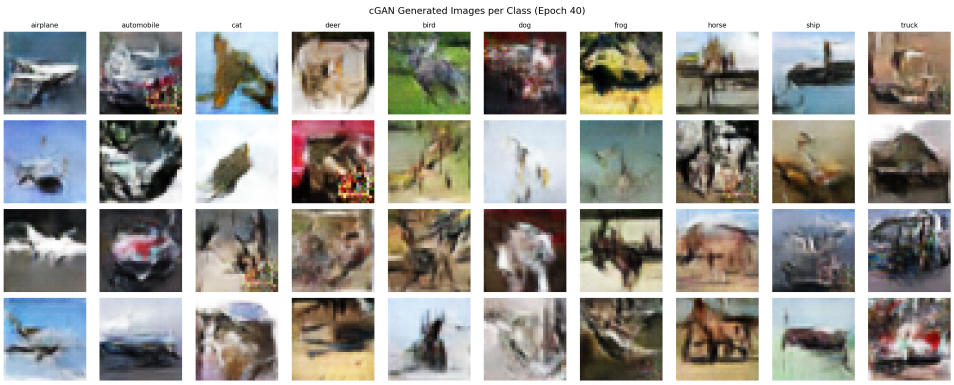

Figure 2: Images generated by the cGAN at epoch 40 checkpoint

generator. The target was to get an FID score of < 50 but the obtained score was around 103.77 due to computational limitations and time constraints. Not only that but the initial image resolution is 32x32 which limits learning and causes the model to not perform as well.

4.2 Qualitative Results

Figure 1 shows the training curve for Generator vs Discriminator loss. Figure 2 shows the images generated by the model with weights corresponding to epoch 40. These images are better than any images at any other checkpoint, hence the final evaluation is done with model at weights corresponding to epoch 40. The generated images show clear class-conditional structure, with classes such as airplane and ship showing recognizable shapes and correct color distributions. Animal classes such as cat and dog are less defined due to their higher intra-class variation.

5 Challenges and Possible Improvements

5.1 Challenges Encountered

Initially, without label smoothing, the discriminator dominated and its loss vanished early on. To fix this, generator was trained twice in a one loop, however this caused the generator to dominate, so, label smoothing to 0.9 with training both once per loop was introduced and the losses could be seen oscillating as in the Training loss in Figure 1. Running all this locally was not possible due to absence of GPU, so the notebook was run on Google Colab with T4 GPU. Furthermore, after 50th epoch it is visible clearly that the generator was more focused on fooling the discriminator than at generating good images, also called mode collapse. Besides all this, FID score target could not be met.

5.2 Possible Improvements

Training for more epochs could potentially improve the results and for a better comparison checkpoints may be saved more frequently. The architecture may not be the best and improvements maybe found with experimentation which could not be done due to time constraints. Having a dataset with higher resolution would also be of huge help to the model. WGAN or other GAN variants might perform better under same conditions.

6 Conclusion

A cGAN was trained on the CIFAR-10 dataset to demonstrate class-conditional training of generator and discriminator to enable the generator to produce images that could make up for class imbalance in a dataset. An initial target FID score was set to 50, but as explained in challenges, an FID score of about 103.77 was achieved. The output serves as a proof of concept; further improvements in this direction may produce state-of-the-art solutions.

References

[1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2014.

[2] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[3] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*, 2014.

[4] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.